Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Synthetic Control and Experimental Design in One Validated Interface: The `mlsynth` Package for Python

Jared Greathouse
Georgia State University

Abstract

Synthetic control and its descendants are a default tool for estimating the effect of a policy, product, or intervention on a few aggregate units. The literature has since fractured into dozens of variants, robust, penalized, factor-model, proximal, forward-selected, and matrix-completion estimators, alongside a newer family of experimental-design methods that pick which units to treat before an intervention runs. Each typically arrives as a separate package with its own data conventions, result format, and inference, or none. This paper introduces `mlsynth`, a strongly typed Python library that places more than forty of them behind one data-preparation step, one validated configuration-and-fit interface, and one standardized result. These are not forty unrelated tools but one family with a shared computational shape, so a unified interface makes them directly comparable and lets the library reach a capability the ecosystem never packaged: experimental design. We demonstrate it with three illustrations, one per regime. The first refits Abadie and Gardeazabal's Basque study with two solvers, a nested search and a provably better bilevel optimizer that no other package carries together, with Cattaneo, Feng, and Titiunik prediction intervals. The second relaxes the no-interference assumption, running four spillover-aware estimators (each otherwise a separate codebase) across two case studies through one dispatcher. The third turns from estimation to design: MAREX, the first packaged synthetic-control experimental design, reproduces Abadie and Zhao's simulation, choosing which units to treat before the experiment and recovering a known effect, then opens onto the broader design family behind the same interface.

Keywords: synthetic control, causal inference, panel data, experimental design, Python.

1. Introduction

A recurring problem in econometrics, marketing measurement, and policy analysis is estimating what would have happened to one unit (a state, a country, a market, a product line) absent

some intervention. Randomized experiments are usually impossible at this aggregation: there is one California, and it either passed an anti-smoking proposition or it did not. The synthetic control method (Abadie and Gardeazabal 2003; Abadie, Diamond, and Hainmueller 2010) answers this by building the missing counterfactual as a weighted average of untreated units, a *synthetic* California assembled from other states whose weighted donors track the treated unit over a pre-intervention window. It is now among the most widely used tools in program evaluation (Abadie 2021), and its original software, the `Synth` package (Abadie, Diamond, and Hainmueller 2011), appeared in this journal.

Fifteen years on, that one method is a research program. The simplex-weighted original sits alongside robust low-rank variants (Amjad, Shah, and Shen 2018), penalized and regularized weights, augmented estimators that debias the in-sample simplex fit with an outcome model (Ben-Michael, Feller, and Rothstein 2021), factor-model and interactive-fixed-effects approaches (Bai 2009; Xu 2017), matrix completion (Athey, Bayati, Doudchenko, Imbens, and Khosravi 2021), the panel-data approach of Hsiao, Ching, and Wan (2012) and its forward selection variant by Shi and Huang (2023), proximal and surrogate methods (Park and Tchetgen 2025; Shi, Li, Yu, Miao, Kuchibhotla, Hu, and Tchetgen 2026; Liu, Tchetgen Tchetgen, and Varjão 2024), and the synthetic-difference-in-differences estimator (Arkhangelsky, Athey, Hirshberg, Imbens, and Wager 2021).

A newer literature inverts the question: instead of building a counterfactual after the intervention, *experimental-design* methods (Abadie and Zhao 2026; Doudchenko, Khosravi, Pouget-Abadie, Lahaie, Lubin, Mirrokni, Spiess, and Imbens 2021) choose which units to treat, and their controls, up front, so a credible counterfactual exists once the experiment ends. Industry moved the same way: Meta's `GeoLift` (Meta Open Source 2024) brought the design-first idea to geographic marketing experiments, building valid geo-experiments on the augmented synthetic control of Ben-Michael *et al.* (2021).

This proliferation carries costs for the practitioner. Each new SCM variant typically ships as its own codebase, usually replication code attached to its paper rather than an installable package (Liu *et al.* 2024; Park and Tchetgen 2025; Amjad *et al.* 2018; Malo, Eskelinen, Zhou, and Kuosmanen 2024). By *packaged* we mean software installable from a standard channel (GitHub, `pip`, CRAN, or Stata's SSC) and called as-is, without moving files into a working directory or reassembling the authors' replication archive by hand.

The pieces scatter across languages even within one lineage: the doubly-robust proximal estimator of Qiu, Shi, Miao, Dobriban, and Tchetgen Tchetgen (2024) is in R, its surrogate-based sibling (Liu *et al.* 2024) in Python; the panel-data regression-control approach is the Stata command `rcm` (Yan and Chen 2022) and the R package `pampe` (Vega-Bayo 2015). A single method fragments too: the original `Synth` surfaced in three syntaxes (unpackaged MATLAB alongside the packaged Stata and R versions), and even the two packaged versions differ, since the R one needs a separate `dataprep` step the Stata command does not. Each package brings its own data format, argument names, notion of a "result", and inference (when it has any). Comparing two estimators on one dataset can mean reshaping the data twice, learning two APIs, and reconciling two result formats by hand. Synthetic difference-in-differences alone ships as the R package `synthdid` and the Stata command `sdid` (Arkhangelsky *et al.* 2021; Clarke, Pailañir, Athey, and Imbens 2023), each with its own syntax.

Worse, the gap between methods and usable software is uneven: advances that would help

analysts often have no packaged implementation at all. A major recent development, the Synthetic Interventions estimator of Agarwal, Shah, and Shen (2026), lets analysts ask, "How would Unit 1, treated with A, have looked under B instead?", a question that recurs in policy analysis, where several treatments may be assigned at once (Sevigny, Greathouse, and Medhin 2023); yet SI has no maintained, packaged implementation. Inference methods for small control pools, such as the leave-two-out refined placebo test of Lei and Sudijono (2025), have no implementation in any language we know of. The predictor-selection method of i Bastida (2022a) has no public implementation. Well-maintained packages exist (Robbins and Davenport 2021; Shi and Huang 2023; Cattaneo, Feng, Palomba, and Titiunik 2025), but much of the new literature ships only as paper-replication scripts, not as software a data scientist can use off the shelf.

All are capable tools, and `mlsynth` critiques none of them. But they span at least two languages, with no shared data format, no shared notion of a result, and no way to run one against another on the same panel without redoing the work each time. A researcher who wanted to compare a simplex control, a panel-data regression, and a synthetic difference-in-differences estimate on one dataset had, until recently, to install several packages, reshape the data several ways, learn several syntaxes, and reconcile several output formats by hand.

This paper introduces `mlsynth`, a Python (Harris, Millman, van der Walt *et al.* 2020) library that addresses this fragmentation. It ships 47 estimators behind one data-preparation routine, one validated configuration-and-fit interface, and one standardized result. Our central argument is that this consolidation is worth having: these are not 47 unrelated tools but one family. As Section 2 makes precise, most of them run the same computation: a treated-unit vector and a donor matrix enter an optimization that returns weights, a counterfactual, and a gap. One interface is therefore not mere packaging convenience. It makes the estimators directly comparable on identical inputs, lets each inherit the same validated data handling and modern inference (conformal and prediction intervals), and lets one library span both the estimation question and the design question the wider ecosystem never packaged together. Concretely, a researcher can now fit a `Synth`-style simplex control, an `rcm`-style panel-data regression, and a `synthdid`-style difference-in-differences design on one data frame, compare them, attach prediction-interval or conformal inference to each, and, in the same syntax, design the experiment a synthetic control will later analyze. That workflow previously spanned several packages across two languages, where available at all.

`mlsynth` aims to make this catalog accessible, free, and as simple to call as possible, and the paper's scope is worth stating plainly. It does not re-derive the forty-odd methods it ships; each has its own optimization, inference theory, and proofs, which belong to the source papers and the per-estimator documentation (for example https://mlsynth.readthedocs.io/en/latest/sdid.html) and which we cite rather than reproduce. Nor is anything special about the three estimators we illustrate below: none is privileged over the dozens we omit, and a different three would have served as well. That arbitrariness is the argument. These methods can be shown side by side, swapped with a one-line edit, only because they share one input contract and one interface, which is what the library exists to provide.

1.1. Related software

Individually, the estimators in `mlsynth` are already well served by existing software. Naming

the tools researchers reach for makes the case for a unified interface concrete and shows what a single validated Python library adds. The landscape is telling: the canonical implementations live in R and Stata, split one method to a package, and the few Python options each cover a narrow slice. We group the representative packages by estimator, note the `mlsynth` equivalent, and report where we have checked the two numerically. Throughout, the point is not that these tools fall short (each is careful and well documented) but that `mlsynth` reaches all of them, and three dozen besides, through one interface.

The original synthetic control method ships as the R package `Synth` (Abadie *et al.* 2011), itself a paper in this journal. It pairs a `dataprep()` routine that reshapes a long panel with a `synth()` routine solving the nested predictor- and donor-weight optimization, plus a few table and plotting functions. It is the reference implementation of the simplex-weighted estimator with predictor matching, which `mlsynth` reproduces as **VanillaSC**.

The optimization `Synth` performs has itself been studied. A recent literature on its numerics, the R package `MSCMT` (Becker and Klößner 2018) and the bilevel-optimization analysis of Malo *et al.* (2024), shows the nested predictor-and-donor problem has a global optimum where the donor weights coincide with a pure outcome fit. `mlsynth` reaches that optimum directly through **VanillaSC**'s outcome-only backend, and the match against the reference is exact: fit to the California tobacco data, **VanillaSC** and the installed `MSCMT` package agree on every donor weight, Utah 0.3939, Montana 0.2318, Nevada 0.2049, Connecticut 0.1091, and on an average treatment effect of −19.51363, to five decimals.

`synth2` (Yan and Chen 2023) re-implements that estimator in Stata and adds the inference practitioners want around it: in-space and in-time placebo tests, a leave-one-out robustness check, and confidence intervals, with publication-ready graphics. In `mlsynth` these are not a separate tool but options on one estimator, **VanillaSC** with `inference = "placebo"` runs the in-space placebo and `inference = "scpi"` attaches the prediction intervals of Cattaneo, Feng, and Titiunik (2021). Two further options go beyond what `synth2` offers, and neither has a packaged implementation in Python: `inference = "lto"` runs the leave-two-out refined placebo test of Lei and Sudijono (2025), far more powerful than the ordinary placebo in small donor pools, and `inference = "ttest"` applies the debiased synthetic-control t-test for the ATT of Chernozhukov, Wuthrich, and Zhu (2025), ported faithfully from the authors' R `scinference`. The same one-word switch thus reaches inference otherwise scattered across separate packages, or, for the refined placebo test, in no package at all.

A second strand replaces the simplex with a regression of the treated unit on the donors, the panel-data approach of Hsiao *et al.* (2012) and its descendants. The Stata package `rcm` (Yan and Chen 2022) implements it with a menu of model-selection routines for choosing which donors enter: best-subset, lasso, and forward and backward stepwise, under AIC, BIC, and cross-validation. The R package `pampe` (Vega-Bayo 2015) implements the same Hsiao-Ching-Wan best-subset estimator and is the standard reference for it; `mlsynth` reproduces its Hong Kong economic-integration application, the original Hsiao *et al.* (2012) study, selecting the same donor set and recovering the published effect path. A distinct refinement, forward-selected PDA, is distributed as the R package `fsPDA` (Shi and Huang 2023), which adds donors greedily by their marginal contribution to pre-treatment fit. `mlsynth` carries this whole strand in **PDA**, whose `method` argument selects the Hsiao-Ching-Wan best-subset fit (`"hcw"`), forward selection (`"fs"`), the lasso, or an ℓ_2 relaxation; on `fsPDA`'s own application, the effect of China's 2012 anti-corruption campaign on luxury-watch imports, **PDA** and `fsPDA` agree

to five decimals, selecting the same three donors and returning a monthly average effect of −0.030896 (−3.09%), matching the −3.09% reported by Shi and Huang (2023) cell-for-cell.

A third strand refines the simplex fit itself. `augsynth` (Ben-Michael *et al.* 2021) is the R implementation of the augmented synthetic control method, which corrects a synthetic control whose pre-treatment fit is imperfect by adding a ridge-penalized outcome model, accommodates auxiliary covariates and staggered adoption, and reports conformal or jackknife-plus inference. Its augmentation is the `augment = "ridge"` option on **VanillaSC**, and the two agree to the sixth decimal: on `augsynth`'s own Kansas tax-cut study, `mlsynth` returns an average effect of −0.029435 for the un-augmented control and −0.040063 for the ridge-augmented fit, matching `augsynth` value-for-value.

A fourth strand changes how donor weights are formed rather than how the simplex is solved. `masc` (Kellogg, Mogstad, Pouliot, and Torgovitsky 2021) is the R implementation of the matching-and- synthetic-control estimator, which interpolates between a nearest-neighbour matching weight and the SC simplex by a cross-validated mixing parameter, trading the extrapolation bias of pure matching against the interpolation bias of pure SC. `microsynth` (Robbins and Davenport 2021), a paper in this journal, calibrates donor weights to match a large set of baseline characteristics exactly and is built for disaggregated, micro-level panels. `mlsynth` carries both as first-class estimators: **MASC** reproduces the Basque ETA-terrorism study of Kellogg *et al.* (2021), the cross-validation selects pure SC, the same Cataluna-plus-Madrid donor pool, and an average effect of −\$585 per capita against the paper's −\$580, and **MicroSynth** reproduces the R package's calibrated-weight solution on the Seattle drug-market study of Robbins and Davenport (2021) to within the table tolerance of that paper.

A final strand replaces the fixed-weight optimization with a Bayesian model. The R package `CausalImpact` (Brodersen, Gallusser, Koehler, Remy, and Scott 2015) fits a Bayesian structural time-series counterfactual, a state-space model with the donors entering as contemporaneous regressors under a spike-and-slab prior, and `mbsts` (Qiu, Jammalamadaka, and Ning 2018) extends that idea to multiple outcomes; in Python, `CausalPy` (PyMC Labs 2024) places a Bayesian synthetic control alongside regression discontinuity, difference-in-differences, and interrupted-time-series designs on a `PyMC` backend. `mlsynth`'s counterpart is **BVSS**, the Bayesian soft-simplex estimator of Xu and Zhou (2025), which puts a spike-and-slab prior over the donors and a soft simplex constraint on their weights, returning full posteriors for the weights and the effect path. The structural time-series tools target a related but distinct counterfactual, a fitted state-space model rather than a weighted combination of donors, so they complement rather than duplicate the SC family.

The Python options for synthetic control itself are fewer and narrower. `SyntheticControlMethods` (Engelbrektson 2020) implements the classic Abadie estimator and its Bayesian and robust variants; `tidysynth` (Dunford 2021) brings the simplex estimator into R's tidyverse with a fluent pipe-based API. Closest to `mlsynth` in spirit is `causaltensor` (Peng, Agarwal, and Shah 2022), a Python library that collects seven matrix-completion and factor-model estimators, difference-in-differences, synthetic difference-in-differences, de-biased convex panel regression, the matrix-completion estimator of Athey *et al.* (2021), and others, behind a common interface. It is the nearest peer to `mlsynth`'s missing-data family (**MCNNM**, **SNN**), but it stops there: it does not carry the simplex, regression-control, penalized, spillover-aware, or experimental-design estimators that make up the rest of the literature.

The pattern across the ecosystem is the friction a unified interface removes. A researcher who wants to weigh a simplex control against a regression-control fit, add augmentation, reach the bilevel optimum, or attach prediction intervals must today cross R, Stata, and Python, learn a different data convention for each, and reconcile as many notions of a result. `mlsynth` is, to our knowledge, the first comprehensive Python library for this family: every estimator named above, and roughly forty more, is reached through one data-preparation step, one configuration-and-fit call, and one standardized result; the numerical matches above are either re-run against the installed reference package or reproduced from the source paper at validation time (Table 1).

Package	Language	Method implemented	In `mlsynth`	Checked
`Synth`	R	simplex SCM, predictor matching	**VanillaSC**	n/a
`synth2`	Stata	simplex SCM, placebo and robustness tests	**VanillaSC**, `inference=`	n/a
`rcm`	Stata	regression control (subset, lasso, stepwise)	**PDA**, `method=`	n/a
`pampe`	R	Hsiao-Ching-Wan panel-data approach	**PDA**, `method="hcw"`	donor set, effect path
`fsPDA`	R	forward-selected PDA	**PDA**, `method="fs"`	-0.030896 (5 dp)
`augsynth`	R	augmented (ridge) SCM, covariates, staggered	**VanillaSC**, `augment="ridge"`	-0.0294/ -0.0401 (6 dp)
`MSCMT`	R	generalized / bilevel SCM	**VanillaSC**, outcome-only backend	installed package (5 dp)
`masc`	R	matching + synthetic control	**MASC**	$-\$585$ vs $-\$580$ (paper)
`microsynth`	R	calibrated SCM for micro-level panels	**MicroSynth**	weights vs R package
`causaltensor`	Python	matrix completion, factor models	**MCNNM**, **SNN**	n/a
`CausalImpact`	R	Bayesian structural time series	**BVSS**	n/a

Package	Language	Method implemented	In `mlsynth`	Checked
`mbsts`	R	multivariate Bayesian structural time series	**BVSS**	n/a
`CausalPy`	Python	Bayesian SC, RD, DiD, ITS (`PyMC`)	**BVSS**	n/a
`tidysynth`	R	simplex SCM (tidyverse API)	**VanillaSC**	n/a
`SyntheticControlMethods`	Python	classic / Bayesian / robust SCM	**VanillaSC**, **CLUSTERSC**	n/a
`mlsynth`	Python	all of the above and roughly forty more	one interface	n/a

Table 1: Representative synthetic-control packages and their `mlsynth` equivalents. The canonical implementations are spread across R and Stata and split one method to a package; the few Python options each cover a narrow slice. The final row is the contribution in one line: a single validated interface, in one language, over a family otherwise spread across three languages and a dozen packages. The "Checked" column records where `mlsynth`'s benchmark suite reproduces the reference implementation or the source paper's published result numerically.

The remainder of the paper is organized as follows. Section 2 develops the shared computational view and the resulting taxonomy of estimators. Section 3 describes the library's software design along the path one call takes: the Pydantic-validated configuration object (Colvin and Pydantic Contributors 2024), the one-package-per-estimator structure, the internal data-preparation routine, and the standardized result hierarchy. Section 4 then illustrates the library by problem regime: one treated unit on Abadie and Gardeazabal's Basque data, fit with two synthetic-control solvers that no other package carries together and with prediction intervals; spillovers, relaxing the no-interference assumption by running four spillover-aware estimators across two case studies through one dispatcher; and experimental design, using MAREX to design an experiment on Abadie and Zhao's simulation, recover a known effect, and situate it in the broader design family. Section 5 concludes, and Section 6 records the computational and reproducibility details.

2. A unified view of synthetic control

2.1. The shared model

Consider a balanced panel of N units observed over T periods. Let y_{it} denote the outcome of unit i in period t. A single treated unit, indexed $i = 1$, receives an intervention at time $T_0 + 1$; the remaining $N-1$ units, the *donor pool*, are never treated. Collect the donor outcomes in the $T \times (N-1)$ matrix $\mathbf{Y}_0$ and the treated unit's outcomes in the T-vector $\mathbf{y}_1$. The first T_0 rows are the pre-intervention window; the remaining $T_1 = T - T_0$ rows are the post-intervention window.

It helps to state the causal target in potential-outcomes terms (Rubin 1974; Imbens and Rubin 2015). Each unit and period has two potential outcomes: $y_{it}(1)$, the outcome were the unit exposed to the intervention, and $y_{it}(0)$, the outcome were it not. Only one is ever realized, the fundamental problem of causal inference (Holland 1986), so for the treated unit after T_0 we observe $y_{1t} = y_{1t}(1)$, while the quantity the analysis needs, the untreated potential outcome $y_{1t}(0)$, is missing. The estimand is the effect on the treated, $\tau_t = y_{1t}(1) - y_{1t}(0)$, and its average over the post-treatment window. The donor pool is never treated, so its outcomes trace the untreated regime; every estimator below borrows that information to impute the missing $y_{1t}(0)$, which the synthetic control $\hat{y}_{1t}$ estimates.

However different their statistical motivations, the estimators in `mlsynth` share one computational shape. Each takes the treated unit's pre-treatment outcome vector $\mathbf{y}_1$ and the donor matrix $\mathbf{Y}_0$, solves an optimization that chooses how to combine the donors, uses the result to impute the missing counterfactual, and reports the gap between the two. Vector, matrix, optimization, weights, plot: that recurring shape, not any one statistical model, is what makes a single interface the right abstraction, and it is the contract that `dataprep` (Section 3.3) and the fit-and-result interface were built to encode.

Why weights that reproduce the pre-treatment vector should also recover the post-treatment counterfactual is a separate, statistical question. The rationale the literature offers is a low-dimensional factor model,

$$y_{it} \;=\; \delta_t \;+\; \boldsymbol{\lambda}_i^{\top}\mathbf{f}_t \;+\; \varepsilon_{it}, \tag{1}$$

where $\mathbf{f}_t$ is an r-vector of unobserved common factors (with r much smaller than N), $\boldsymbol{\lambda}_i$ is unit i's loadings, δ_t is a common time effect, and ε_{it} is idiosyncratic noise. Units differ only through the handful of latent loadings $\boldsymbol{\lambda}_i$, so if the treated unit's loadings are a combination of the donors', weights that reproduce its pre-treatment outcomes also reproduce its loadings, and hence its untreated path after treatment. We take this as the standard justification, refer readers to the source papers, and note that the library does not commit to it: several members of the family relax or replace it. The synthetic control

$$\hat{y}_{1t} \;=\; \mathbf{Y}_{0,t}\,\mathbf{w}, \qquad t > T_0, \tag{2}$$

is then an estimate of the missing counterfactual, and the treatment effect in period t is the gap $\tau_t = y_{1t} - \hat{y}_{1t}$.

For almost every estimator in the library, the weights solve one common optimization program. Writing $\mathbf{Y}_0$ and $\mathbf{y}_1$ now for their pre-treatment (T_0-row) blocks, the donor weights solve

$$\hat{\mathbf{w}} \;\in\; \underset{\mathbf{w}\,\in\,\mathcal{C}}{\arg\min}\;\; \mathcal{L}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) \;+\; \mathcal{P}(\mathbf{w}) \qquad \text{subject to} \qquad \mathcal{B}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) \;\le\; \tau. \tag{3}$$

Here $\mathcal{L}$ is a loss measuring pre-treatment fit, $\mathcal{P}$ is a penalty that shapes the weights, $\mathcal{C}$ is the admissible set the weights must lie in, and $\mathcal{B} \le \tau$ is an optional set of balance conditions

with relaxation level τ; either the loss or the balance operator may be absent, but not both. Classical synthetic control (Abadie *et al.* 2010) is the specialization with a squared-error loss $\mathcal{L} = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2$, no penalty ($\mathcal{P} = 0$), no separate balance constraint, and the admissible set fixed to the unit simplex $\mathcal{C}_\Delta = \{\mathbf{w} \geq \mathbf{0} : \mathbf{1}^\top \mathbf{w} = 1\}$, which yields an interpretable, sparse convex combination of donors. The rest of the family is, to a first approximation, a catalog of other choices for the four objects in Equation 3. The most visible is the admissible set $\mathcal{C}$ itself: relaxing the simplex to merely non-negative weights drops the sum-to-one requirement; an affine constraint, $\mathbf{1}^\top \mathbf{w} = 1$ alone, permits signed weights and extrapolation beyond the donors' range; a unit box confines each weight to $[0, 1]$ on its own; and dropping the constraint altogether leaves an ordinary regression of the treated unit onto the donors.

Penalties $\mathcal{P}$ supply the rest: an ℓ_2 term gives ridge-augmented weights, an ℓ_1 term gives a sparse selection of donors, and an intercept b_0, folded in by augmenting $\mathbf{Y}_0$ with a column of ones and left unpenalized, absorbs level differences between the treated unit and its donors. Moving the fit from the objective into the constraint $\mathcal{B} \leq \tau$ recovers the balancing and relaxation estimators, where one minimizes a norm of $\mathbf{w}$ subject to approximate moment matching.

What then separates the estimators is *how* the weights $\mathbf{w}$ (or, more generally, the projection onto the donor span) are obtained, and what is assumed about Equation 1. Beyond the choice of admissible set and penalty in Equation 3, members of the family differ in how they treat the donor matrix itself:

- Penalized and ridge-augmented weights trade a small amount of in-sample fit for stability when donors are collinear (Ben-Michael *et al.* 2021).
- Principal-component and robust variants first denoise $\mathbf{Y}_0$ by truncating its singular-value spectrum (estimating $\mathbf{f}_t$ directly) before fitting the weights (Amjad *et al.* 2018), which is exactly the low-rank reading of Equation 1.
- Forward-selection and best-subset approaches, following the panel-data method of Hsiao *et al.* (2012), choose a small set of donors by an information criterion rather than weighting all of them.
- Factor and interactive-fixed-effects estimators fit Equation 1 explicitly (Bai 2009; Xu 2017), and matrix-completion methods recover the full low-rank outcome matrix (Athey *et al.* 2021).

Seen this way, the catalog is a set of choices along a few axes (the constraint on the weights, whether the donor matrix is denoised first, whether donors are selected or weighted, how the common time structure is handled), not a list of unrelated algorithms. Every one still consumes the same inputs ($\mathbf{Y}_0$, $\mathbf{y}_1$, T_0) and returns the same output, a counterfactual path and its gap, which is why the whole family fits behind one interface.

2.2. From estimation to design

The same pipeline runs in reverse for experimental design. If a credible synthetic control needs donors whose pre-treatment outcomes can reconstruct the treated unit, then before running an experiment one can ask which candidate units *should* be treated so the remaining units form a good donor pool, and how long the experiment must run to detect a given effect size. Design-first methods (Doudchenko *et al.* 2021) solve exactly this, turning the estimation

machinery around: the fit quality that validates a synthetic control after the fact becomes the objective that selects treatment units beforehand. `mlsynth` treats design as a first-class peer of estimation rather than a separate concern, and Section 4.3 demonstrates it.

3. Software design

The library is organized so that each piece teaches the next. A user writes one configuration, hands it to one estimator, and reads one result; in between, the data are prepared, internally, identically, by one routine the user never calls. We follow that path (config, estimator, data preparation, result) because it is also the order in which the design builds on itself. Two commitments run through all four. Outwardly, every estimator presents the same surface, so learning one teaches every other. Inwardly, every estimator is its own self-contained package, which keeps more than forty of them maintainable.

3.1. One config

Each estimator is driven by a dedicated configuration object built on Pydantic (Colvin and Pydantic Contributors 2024), a runtime type-validation library. Configurations forbid unknown fields, document every option, and fail early with a typed error when an input is malformed, a missing treatment column, a treatment date outside the sample, an empty donor pool. There are no free-form keyword arguments. The usage pattern is uniform: construct a configuration (here as a plain dictionary, which Pydantic coerces and validates), pass it to the estimator class, and call `fit()`.

```
from mlsynth import VanillaSC

config = {
    "df": panel,            # a long/tidy pandas DataFrame
    "outcome": "cigsale",   # the outcome column
    "treat": "treat",       # 0/1 treatment indicator
    "unitid": "state",      # the unit column
    "time": "year",         # the time column
}
results = VanillaSC(config).fit()
```

The five required fields name the parts of the panel. `df` is the tidy long data frame itself, one row per unit and period. `outcome` names the column to be explained, the variable whose treated-minus-counterfactual gap is the object of interest. `treat` names a 0/1 indicator equal to one for the treated unit in its post-intervention periods and zero everywhere else; from it alone the package reads which unit is treated and when, so the intervention date and the pre/post split need never be stated separately. `unitid` and `time` name the columns that identify the unit and the period. Everything past these five is method-specific and optional, each carrying its own default.

Switching estimators means changing the class name and, at most, a handful of method-specific options; the data, the column names, and the call shape do not move. This is the concrete payoff of the unified view in Section 2.

Under the surface, this uniformity is enforced by type rather than convention. Every configuration descends from one of two base models. Observational estimators extend `BaseEstimatorConfig`, which declares the five required fields above; the experimental-design family extends `BaseMAREXConfig`, identical but for one telling omission, it requires no `treat` column, because a design chooses whom to treat rather than reading it from the data. That split in the type system is the two-family results contract of Section 3.4 seen from the input side. Both bases forbid any field they do not recognize and validate the panel on construction, and a specialized estimator extends its base with its own typed, documented options.

```
from mlsynth.config_models import BaseMAREXConfig
from mlsynth.utils.marex_helpers.config import MAREXConfig

design_fields = set(MAREXConfig.model_fields) - set(BaseMAREXConfig.model_fields)
n_design_fields = len(design_fields)
```

The design estimator MAREX is the fullest example. Its `MAREXConfig` inherits the design base and layers 35 further fields on top (a cluster column, per-stratum quotas, a budget, an adjacency matrix, lists of units to force into or out of treatment), each a declared field with a description and a validator, none of them a free-form keyword.

Because the fields are typed and the bases forbid the unknown, malformed input is rejected at construction, before any computation, and with a translated `mlsynth` exception rather than a raw trace. Naming an outcome column the data does not have is caught immediately:

```
import pandas as pd
from mlsynth import VanillaSC
from mlsynth.exceptions import MlsynthDataError

panel = pd.DataFrame({"state": ["A", "A", "B", "B"], "year": [1, 2, 1, 2],
                      "cigsale": [10.0, 9.0, 8.0, 7.0], "treat": [0, 1, 0, 0]})
try:
    VanillaSC({"df": panel, "outcome": "sales",        # no 'sales' column
               "treat": "treat", "unitid": "state", "time": "year"})
except MlsynthDataError as err:
    print(f"{type(err).__name__}: {err}")
```

```
MlsynthDataError: Missing required columns in DataFrame 'df': sales
```

Every estimator inherits the same discipline, so across the whole catalog a malformed analysis fails the same way: fast, typed, and legible.

Because a configuration is plain, validated data rather than live program state, the specification of an analysis is itself serializable. Everything in it but the DataFrame (the column names and every method-specific option) can be written to a JSON or YAML file, version-controlled, and loaded back to drive a fit. The record of *what* analysis was run thus lives separately from the data it ran on, which keeps to its own tabular format (CSV or Parquet); each can be inspected and diffed on its own. A reproducible study can therefore ship as a small text specification beside its data, with no bespoke script standing between the two. Concretely, the configuration above is a few lines of YAML,

```
# study.yaml: a complete, shareable analysis specification
```

```
estimator: VanillaSC
outcome:   cigsale
treat:     treat        # 0/1 treatment indicator
unitid:    state
time:      year
```

which `mlsynth` reads back into a ready-to-fit estimator, resolving the named method and attaching the data at load time:

```
from mlsynth import load_spec

est = load_spec("study.yaml", df=panel)   # resolves VanillaSC, attaches the panel
results = est.fit()
```

The library exposes the writing side symmetrically, as `save_spec`, so a fitted configuration can be exported to a record and shared.

That this takes only a few lines, and works for every estimator alike, is a dividend of the shared contract, not per-method plumbing: because a configuration is data, the estimator is named in its own record, and one `fit()` runs whatever loads, the uniformity that makes the catalog comparable also makes its analyses portable.

3.2. One estimator

That uniform surface is only affordable because the source is partitioned the same way the method catalog is. Each estimator is one package. A thin class in `estimators/<name>.py` does nothing but validate its configuration, hand off to a pipeline, translate any failure into the library's typed error vocabulary, and return the standardized result; **VanillaSC** is seventy-odd lines and holds no algorithm of its own. The work lives beside it in a helper package, `utils/<name>_helpers/`, split by role, typically a setup step that prepares the data, the numerical pipeline, the data structures, a plotter, and whatever modules are specific to the method. The configuration object lives in that same package, in its own `config.py`, next to the code it governs, and is re-exported centrally so existing imports keep resolving.

The motivation is mundane and decisive. The alternative, the project's own earlier design, is a handful of shared modules that every estimator edits in common; a single configuration file had grown toward several thousand lines, becoming longer and riskier to touch with each method added. Giving every method its own package is the remedy. A contributor adding or revising an estimator works inside one directory: the change cannot reach another estimator's code, two people editing different estimators never collide, and the blast radius of a mistake is one package. The same boundary serves the reader. The unit one must hold in mind to follow a method, its options, its validation rules, its numerics, and its tests, is exactly one directory, not a thread chased across a monolithic configuration module, a monolithic estimator module, and a shared helper file.

What stays shared is deliberately small and cross-cutting. Every configuration inherits one base, `BaseEstimatorConfig`, which fixes the common `df / outcome / treat / unitid / time` surface and forbids unknown fields, so a method's own `config.py` declares only what is genuinely specific to it. Every failure is raised in one hierarchy, `MlsynthConfigError`,

`MlsynthDataError`, and `MlsynthEstimationError` beneath a common `MlsynthError`, so that user code catches the same types no matter which estimator raised them. And when a method is really a family of related variants, the package gains a dispatcher rather than the library gaining several near-duplicate estimators: **SPILLSYNTH** selects among five spillover-adjustment schemes through a single `method` argument, each isolated in its own subpackage. The rule scales the catalog without scaling the coupling.

3.3. One dataprep

Inside that thin estimator class, the first step is data preparation, the one part of the library the analyst never touches. `mlsynth` asks only for a clean, tidy panel, a long data frame, one row per unit and period, prepared before any estimator sees it. Ingestion is then routed through a single internal routine, `dataprep()`, which the user never calls. It takes the tidy frame together with the names of its outcome, unit, time, and treatment columns and returns a canonical bundle, the wide outcome matrix, the treated vector $\mathbf{y}_1$, the donor matrix $\mathbf{Y}_0$, and the pre- and post-treatment period counts T_0 and T_1. Every estimator consumes that same bundle, so every data frame is pivoted, aligned, and validated the same way, every single time, in one place that is written and tested once. What varies is only what the analysis requires (whether covariates are supplied, say), never the mechanics of getting a data frame into an estimator.

The implication worth dwelling on is what the user does *not* have to specify. A single binary treatment indicator carries all the structure `dataprep()` needs: the unit whose indicator ever turns on is the treated unit, the period in which it first turns on is $T_0 + 1$, and everything before that is the pre-treatment window. The user never names the treated unit, never states the intervention date, and never declares the pre/post split; these are read off the indicator. The same convention scales without new arguments: when several units are treated at different times, `dataprep()` cohorts them by their individual adoption dates automatically.

This is a deliberate contrast with the surrounding ecosystem, and it shows first as sheer simplicity at the call site. To get from a data frame to an estimated, plotted result with the widely used `scpi` package (Cattaneo *et al.* 2025), the analyst imports six separate functions and chains four of them by hand, `scdata` to prepare and reshape the data, then `scest` to estimate, `scpi` to do inference, and `scplot` to draw the result, with the data-preparation step, `scdata`, squarely the analyst's job. `tidysynth` (Dunford 2021), the most ergonomic of the R options, still asks for a pipeline of four verbs, `synthetic_control`, `generate_predictor`, `generate_weights`, and `generate_control`, and then separate `grab_*` and `plot_*` calls to recover the weights, the counterfactual, and the figures. The forward-selection `fsPDA` implementation (Shi and Huang 2023) requires the data to be pre-pivoted into wide treated-versus-donor matrices before it will run at all. In `mlsynth` the analyst imports one estimator, builds one configuration, and calls `fit()` once; there is no preparation function to find, import, configure, or call. Coding the indicator column *is* the specification, and the same long data frame drives every one of the more than forty estimators unchanged.

3.4. One result

Every `fit()` returns a typed result object drawn from a single hierarchy. Observational

estimators return an effect result; design methods return a design result whose report is itself an effect result. Both populate the same standardized sub-objects, estimated effects, fit diagnostics, the observed and counterfactual time series, donor weights, and inference, as typed fields rather than ad hoc dictionaries. Reading a result is therefore the same regardless of which estimator produced it:

```
res = VanillaSC(config).fit()

res.effects.att                          # average effect on the treated
res.time_series.counterfactual_outcome   # the synthetic control path
res.weights.donor_weights                # the fitted donor weights
res.inference.p_value                    # inference, on the same object
```

The same fields, read off a real fit, return the numbers themselves. Here is **VanillaSC** on the Basque Country data (Abadie and Gardeazabal 2003), using the outcome-only backend and the debiased synthetic-control t-test of Chernozhukov *et al.* (2025):

```
pd.Series({
    "effects.att":         round(res.effects.att, 4),          # average effect
    "effects.att_percent": round(res.effects.att_percent, 2),  # as a percentage
    "inference.p_value":   round(res.inference.p_value, 4),    # debiased t-test
    "inference.ci_lower":  round(res.inference.ci_lower, 4),
    "inference.ci_upper":  round(res.inference.ci_upper, 4),
    "inference.method":    res.inference.method,
})
```

```
effects.att                                                  -0.6915
effects.att_percent                                             -8.1
inference.p_value                                              0.042
inference.ci_lower                                           -1.2561
inference.ci_upper                                           -0.0589
inference.method       debiased SC t-test (Chernozhukov-Wuthrich-Zhu ...
dtype: object
```

Because the result shape is fixed, downstream code (plotting, tables, comparison across estimators) is written once and works for every method. The illustrations below read effects, time series, and inference off this common object without any estimator-specific glue.

4. Applications

The next three subsections put the library to work, one per problem regime: a single treated unit, spillovers, and experimental design. Each is a self-contained, reproducible transcript, and together they show that moving between regimes is a change of estimator against an unchanged data frame, not a change of tool.

4.1. A single treated unit

We begin with the other study that made the method famous. Abadie and Gardeazabal (2003)

built a synthetic Basque Country to estimate what ETA's terrorism campaign cost the regional economy, tracking per-capita GDP against a weighted combination of other Spanish regions.

The example also lets us look inside the *optimization* step of Section 2, because a synthetic control's weights are harder to compute than they look. The classical specification chooses predictor weights and donor weights together, an outer search over the predictors wrapped around an inner fit for the donors, and Malo *et al.* (2024) show this nested problem is a bilevel program whose optimum is typically a corner solution that data-driven nested solvers can fail to reach. Two solver families address it. The nested differential-evolution search of the R package `MSCMT` (Becker and Klößner 2018) is the careful, widely used implementation of Abadie and Gardeazabal's original procedure; the bilevel solver of Malo *et al.* (2024) reaches the global optimum but ships only as replication code, in no installable package. `mlsynth` carries both as backends of one estimator, **VanillaSC**, one `backend` keyword apart, so they can be run on the same panel and compared, which no other package permits.

```
# Abadie-Gardeazabal's Basque panel (the MSCMT-transformed spec), treated from
# 1970, with their thirteen predictors each averaged over its own window.
basque = pd.read_csv(find_data("basque_mscmt.csv"))
basque["treat"] = ((basque["regionname"] == "Basque Country (Pais Vasco)")
                   & (basque["year"] >= 1970)).astype(int)

schooling = ["school.illit", "school.prim", "school.med", "school.higher", "invest"]
sectors = ["sec.agriculture", "sec.energy", "sec.industry", "sec.construction",
           "sec.services.venta", "sec.services.nonventa"]
covariates = schooling + ["gdpcap"] + sectors + ["popdens"]
windows = ({c: (1964, 1969) for c in schooling}
           | {c: (1961, 1969) for c in sectors}
           | {"gdpcap": (1960, 1969), "popdens": (1969, 1969)})

ag_base = dict(df=basque, outcome="gdpcap", treat="treat", unitid="regionname",
               time="year", covariates=covariates, covariate_windows=windows,
               fit_window=(1960, 1969), inference="scpi", alpha=0.10,
               display_graphs=False)
```

One problem, two solvers

We fit the Abadie-Gardeazabal specification twice, changing only the solver. The first call passes `backend="mscmt"`, the nested differential-evolution search, which reproduces the `MSCMT` vignette value-for-value; the second passes `backend="malo"`, the bilevel solver, which reproduces the `scm.corner` reference of Malo *et al.* (2024). Both attach the Cattaneo *et al.* (2021) prediction intervals through the same `inference="scpi"` switch.

```
mscmt = VanillaSC({**ag_base, "backend": "mscmt", "canonical_v": "min.loss.w",
                   "mscmt_maxiter": 400, "mscmt_popsize": 20, "seed": 42}).fit()
malo = VanillaSC({**ag_base, "backend": "malo", "seed": 0}).fit()       # one keyword

years = np.array(sorted(basque["year"].unique()))
pre = (years >= 1960) & (years <= 1969)
```

```
observed = (basque[basque["regionname"] == "Basque Country (Pais Vasco)"]
            .sort_values("year")["gdpcap"].to_numpy())

solvers = {"MSCMT nested solver": mscmt, "Malo bilevel solver": malo}
cmp = compare_counterfactuals(solvers, observed=observed, time=years)

# Stored, not recomputed: effects.att and the full-pre fit_diagnostics.rmse_pre.
att_mscmt = cmp.summary.loc["MSCMT nested solver", "att"]
att_malo = cmp.summary.loc["Malo bilevel solver", "att"]
gap_pct = 100 * (att_malo - att_mscmt) / att_mscmt
allpre_mscmt = cmp.summary.loc["MSCMT nested solver", "pre_rmse"]
allpre_malo = cmp.summary.loc["Malo bilevel solver", "pre_rmse"]

# Fit-window RMSE: the outcome loss over 1960-1969, the quantity each solver
# minimizes (a tighter window than the stored all-pre rmse_pre above).
def fit_rmse(res):
    cf = np.asarray(res.time_series.counterfactual_outcome, dtype=float)
    return float(np.sqrt(np.mean((observed[pre] - cf[pre]) ** 2)))
```

```
fig, ax = plt.subplots(figsize=(5, 3.2))
cmp.plot(ax=ax, dodge=0.5,           # one call: both counterfactuals + CFT bands
         colors={"MSCMT nested solver": "C0", "Malo bilevel solver": "C3"},
         styles={"MSCMT nested solver": "--", "Malo bilevel solver": "-."},
         legend=False)
ax.axvline(1970, color="0.4", lw=1.0)
ax.set_xlabel("year"); ax.set_ylabel("GDP per capita (000s USD)")
ax.legend(fontsize=6.5, loc="upper left"); fig.tight_layout(); plt.show()
```

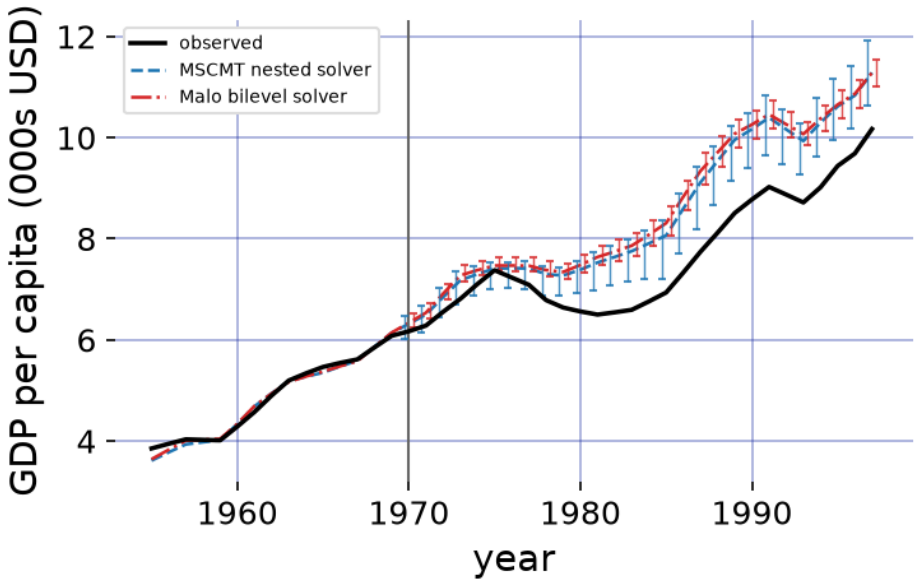

Figure 1: The Abadie-Gardeazabal Basque study fit twice, changing only the solver. Observed Basque per-capita GDP (black) against the synthetic control from the MSCMT nested differential-evolution search (blue, dashed) and the Malo et al. bilevel solver (red, dash-dotted), each carrying its Cattaneo-Feng-Titiunik 90% prediction interval as per-period error bars over the post-treatment window (dodged for legibility). The solid vertical line marks 1970, the start of the treatment window; the solvers fit the pre-period almost identically yet rest on different donor weights (Figure 2).

The two solvers build different synthetic Basque Countries from the same donor pool (Figure 2). The nested search puts most of its weight on Cataluna; the bilevel solver drops Cataluna entirely, promotes Madrid to the largest donor, and adds Rioja. They are solving the same problem, and Malo *et al.* (2024)'s point is that the nested search need not reach its optimum, which is what happens here: the bilevel solution attains a lower pre-treatment fit loss over the matching window (RMSE 0.0642 against 0.0655, and 0.0823 against 0.0969 over the full pre-period, the result's stored `pre_rmse`). The better fit moves the headline number. The nested solver puts the average GDP shortfall from terrorism at 0.89 thousand USD per capita; the bilevel optimum, fitting the pre-period better, puts it 11% larger at 0.98, and with a tighter prediction interval that the looser fit would not earn. Malo *et al.* (2024) demonstrate this failure on the California tobacco study; here it recurs on the Basque study, the field's other canonical application, in a side-by-side comparison only possible because both solvers sit behind one interface.

Both solutions live under the same `weights.donor_weights` field, and Figure 2 plots them side by side.

```
def positive_weights(res):
    return {str(k): float(v) for k, v in res.weights.donor_weights.items()
            if abs(v) > 1e-3}

wm, wl = positive_weights(mscmt), positive_weights(malo)
```

```
donors = sorted(set(wm) | set(wl), key=lambda r: -(wm.get(r, 0) + wl.get(r, 0)))
short = lambda r: r.split(" (")[0]
x = np.arange(len(donors)); bw = 0.38

fig, ax = plt.subplots(figsize=(5, 2.8))
ax.bar(x - bw / 2, [wm.get(r, 0) for r in donors], bw, color="C0", label="MSCMT nested")
ax.bar(x + bw / 2, [wl.get(r, 0) for r in donors], bw, color="C3", label="Malo bilevel")
ax.set_xticks(x); ax.set_xticklabels([short(r) for r in donors])
ax.set_ylabel("donor weight"); ax.legend(fontsize=7); fig.tight_layout(); plt.show()
```

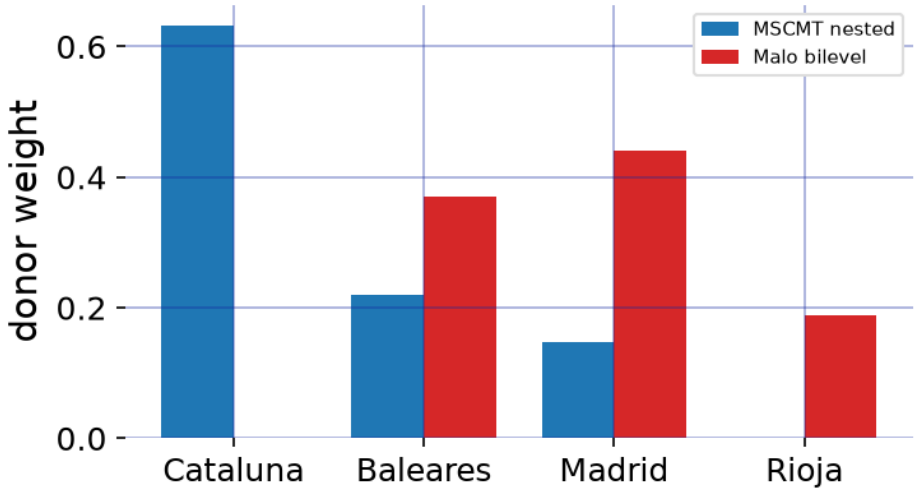

Figure 2: Donor weights from the two solvers (`weights.donor_weights`), regions with negligible weight omitted. The nested search (blue) and the bilevel search (red) select different regions, Cataluna versus Rioja, and weight their shared donors, Madrid and Baleares, differently, for the same treated unit.

That the comparison is controlled is the point. Only the solver changes; the data frame, the data-preparation contract, and the result schema are held fixed, so the disagreement between a widely used nested solver and the provably better bilevel one is attributable to the solver alone rather than to two reconciled implementations. That attribution is sound only because `mlsynth` hosts both, each cross-validated against its reference (the R package `MSCMT` and Malo *et al.* (2024)'s `scm.corner`), behind one estimator and one `backend` keyword. The same one-field edit reaches the rest of the catalog: `SDID` adds unit and time weights to a two-way fixed-effects fit (Arkhangelsky *et al.* 2021), `CLUSTERSC` de-noises the donor matrix before fitting (Amjad *et al.* 2018), each on the unchanged `ag_base` configuration. A change of method is a change of one field, not a migration across packages and languages.

4.2. Spillovers

The illustration so far treats the donor pool as clean: untreated units are assumed unaffected by the treatment, the stable-unit-treatment-value assumption (SUTVA) that synthetic control inherits from the broader causal-inference literature. That assumption is often indefensible.

When California raised its cigarette tax, some sales simply crossed the border into Nevada, so a neighbouring state's outcomes were themselves moved by the treatment; using it as a donor contaminates the counterfactual. For most of the method's history the remedy was subtractive (drop the units you suspect, as Abadie *et al.* (2010) do), trading a SUTVA violation for a smaller, weaker comparison. A more recent literature instead keeps the units and models the spillover: Cao and Dowd (2023), Di Stefano and Mellace (2024), Melnychuk (2024), and Grossi, Mariani, Mattei, Lattarulo, and Öner (2025) each estimate the direct effect and the leakage jointly.

These four estimators arrived as four separate codebases, in another language, with four data conventions. `mlsynth` exposes them through one dispatcher, **SPILLSYNTH**, selected by a `method` keyword; the treated unit, the donor pool, and the units suspected of absorbing spillover are declared once. The demonstration we want is therefore not of any single estimator but of the consolidation: a common interface makes four spillover-aware methods commensurable, so their estimates can be compared on the same panel rather than reconciled across four output formats. We run all four on the California tobacco panel, with the dozen neighbours Abadie *et al.* (2010) discard reinstated as suspected spillover recipients, and on West Germany's 1990 reunification (Abadie, Diamond, and Hainmueller 2015), with Austria, France, and the Netherlands as the recipients, plotting each method's spillover-adjusted counterfactual against the observed treated unit.

```
adh_covariates = ["trade", "infrate", "industry", "schooling",
                  "invest60", "invest70", "invest80"]
cases = [
    dict(name="Proposition 99", file="prop99_with_dc.csv", span=(1970, 2000),
         outcome="cigsale", unit="state", time="year", treated="California",
         t0=1989, ylab="cigarette sales (packs p.c.)", covariates=None,
         affected=["Alaska", "Arizona", "District of Columbia", "Florida",
                   "Hawaii", "Massachusetts", "Maryland", "Michigan",
                   "New Jersey", "Nevada", "New York", "Oregon", "Washington"]),
    dict(name="Reunification", file="scpi_germany.csv", span=(1960, 2003),
         outcome="gdp", unit="country", time="year", treated="West Germany",
         t0=1990, ylab="GDP p.c. (000s USD)", covariates=adh_covariates,
         affected=["Austria", "France", "Netherlands"]),
]
methods = {"cd": ("C0", "-"), "iscm": ("C3", "--"),
           "iterative": ("C1", "-."), "grossi": ("C4", ":")}

fig, axes = plt.subplots(1, 2, figsize=(7, 3))
records = []
for ax, case in zip(axes, cases):
    df = pd.read_csv(find_data(case["file"]))
    df = df[df[case["time"]].between(*case["span"])].copy()
    df["treat"] = ((df[case["unit"]] == case["treated"])
                   & (df[case["time"]] >= case["t0"])).astype(int)
    cfg = dict(df=df, outcome=case["outcome"], treat="treat", unitid=case["unit"],
               time=case["time"], affected_units=case["affected"],
               display_graphs=False)
```

```
    years = np.array(sorted(df[case["time"]].unique()))
    obs = (df[df[case["unit"]] == case["treated"]]
           .sort_values(case["time"])[case["outcome"]].to_numpy())
    specs = {}
    for m in methods:
        kw = {**cfg, "method": m}
        if m == "iscm" and case["covariates"]:
            kw["covariates"] = case["covariates"]      # only iscm matches on covariates
        fit = SPILLSYNTH(kw).fit()                      # the one line that varies
        sub = getattr(fit, m)
        pre = (sub.a[0] + sub.B[0] @ fit.inputs.Y_pre if m == "cd"
               else sub.treated_synthetic_pre)
        full = np.concatenate([np.asarray(pre), np.asarray(sub.counterfactual_sp)])
        specs[m] = {"counterfactual": full, "att": fit.att, "time": years}
    cmp = compare_counterfactuals(specs, observed=obs, time=years)  # one overlay
    cmp.plot(ax=ax, colors={m: methods[m][0] for m in methods},
             styles={m: methods[m][1] for m in methods}, legend=False)
    for m in methods:
        records.append({"case": case["name"], "method": m,
                        "ATT": cmp.summary.loc[m, "att"]})
    ax.axvline(case["t0"], color="0.4", ls="-", lw=1.0)
    ax.set_title(case["name"], fontsize=8)
    ax.set_xlabel("year"); ax.set_ylabel(case["ylab"], fontsize=8)
axes[0].legend(fontsize=6, ncol=2)
fig.tight_layout()
plt.show()
```

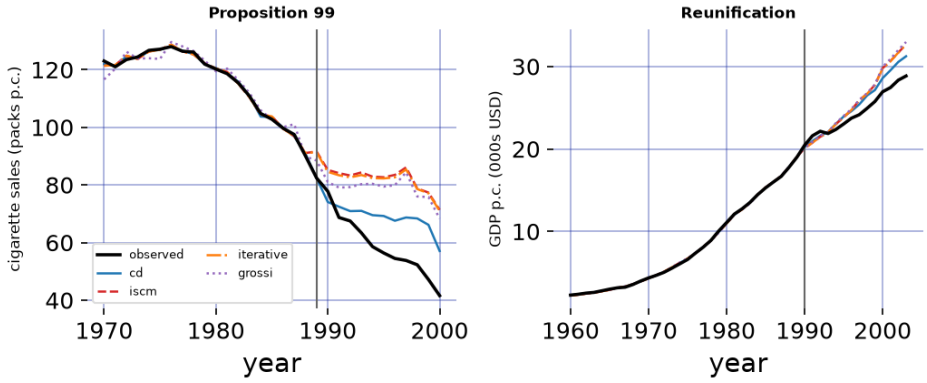

Figure 3: Four spillover-aware estimators on two case studies, reached by changing one `method` keyword in a loop. Each panel plots the observed treated unit (black) against each method's spillover-adjusted synthetic counterfactual. On Proposition 99 (left) the methods disagree substantially, since spillover into the reinstated neighbours is large; on German reunification (right) they nearly coincide, since the declared spillover is small. The spread within a panel is a cross-method sensitivity check that the shared interface makes free.

```
att = pd.DataFrame(records)
```

```
att_units = {"Proposition 99": "ATT (packs p.c.)", "Reunification": "ATT (000s USD)"}

fig, axes = plt.subplots(1, 2, figsize=(7, 2.8))
for ax, case in zip(axes, att_units):
    sub = att[att["case"] == case].set_index("method").reindex(methods)
    ax.bar(np.arange(len(methods)), sub["ATT"],
           color=[methods[m][0] for m in methods])
    ax.axhline(0, color="0.4", lw=0.8)
    ax.set_xticks(np.arange(len(methods))); ax.set_xticklabels(list(methods))
    ax.set_title(case, fontsize=8); ax.set_ylabel(att_units[case], fontsize=8)
fig.tight_layout(); plt.show()
```

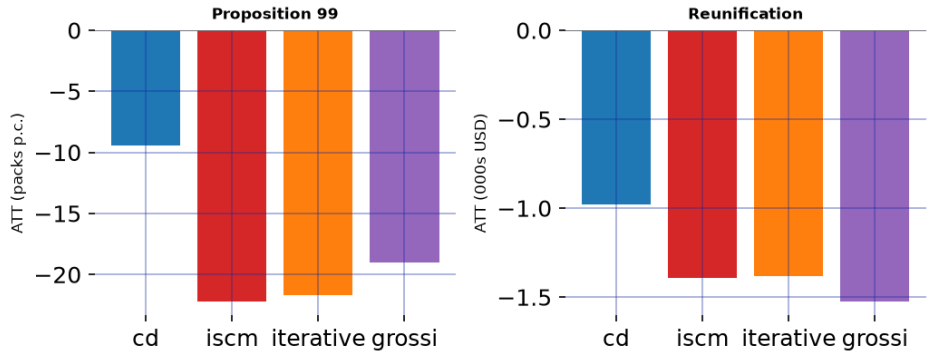

Figure 4: Average treatment effect on the treated for each spillover-aware estimator on each case study (the `att` field). One panel per case because the outcomes are in different units; within a panel the bars share an axis, so the cross-method spread is read directly. The Proposition 99 methods disagree by more than a factor of two; the reunification methods nearly agree, tracking how much spillover each case has to model.

The two panels make the same point from opposite directions. On Proposition 99 the four methods disagree by more than a factor of two, because the treatment genuinely leaks into the reinstated neighbours and each estimator apportions that leakage differently; on reunification, where only three modest neighbours are declared affected, the corrections are small and the four counterfactuals nearly coincide. The disagreement is information, not defect: its size tracks how much spillover there is to model, and it is a quantity the reader can trust because every method saw the same prepared panel and returned the same result schema, so the spread measures the methods rather than four hand-aligned outputs. As with the single-unit illustration, `mlsynth` takes no position on which estimator is correct, that judgement belongs to the analyst and the application. What the package supplies is the prior thing that makes the judgement possible: a commensurable reading of all of them, on any of your panels, behind one interface.

4.3. Experimental design

The first two illustrations were *retrospective*. The treatment had already happened, and the estimator's job was to reconstruct, after the fact, the counterfactual path of a unit that was

treated whether or not a good comparison existed for it. The third regime inverts the order of operations. Here the experiment has not run yet, and the question is one of *design*: of all the units we could treat, which ones should we, and which should we hold back as controls, so a credible comparison exists once the experiment is over? The data are the same shape as before (a panel of units over time) but the decision has moved from after the intervention to before it, and that makes it a genuinely different problem. In the familiar retrospective setting the treated unit is given and the analyst is a price-taker on assignment; here the assignment is the thing being chosen, and choosing it well is what makes the eventual estimate trustworthy.

A recurring question in marketing measurement is what a firm's advertising actually buys: when we spend on paid media, how many additional sales does that spending cause? One clean way to answer it is a go-dark test. The firm turns paid media off in a few of its markets for a fixed window, keeps it on everywhere else, and asks what the darkened markets would have sold had the spending continued. The revenue the advertising was generating is that counterfactual, how much the firm would be selling if it had never cut funding to zero, minus what the dark markets actually sold.

The hard part is choosing which markets go dark. A market here is a metro advertising region, the smallest unit a firm can darken, and a firm usually has only a handful of them, large and quite different from one another. Splitting them into dark and lit groups at random is unbiased on average, but a single random split can easily put the biggest markets on one side, so the two groups start from different sales levels and the comparison is compromised before the test begins; you run the experiment once, so you are stuck with whatever that one split gave you. MAREX (Abadie and Zhao 2026) instead chooses the dark and lit markets up front, so that each group's pre-launch sales reproduce the whole market's. The paper proves this deliberately non-randomized design sharply reduces the bias of the effect you later estimate, and to our knowledge `mlsynth` is the first package to make the method available off the shelf.

Choosing the two groups is combinatorial: each market may join the dark (treated) group, the lit (control) group, or neither, and the two synthetic units must each reproduce the population predictor mean. Write $\mathbf{x}_j$ for market j's predictors (its pre-launch sales together with a handful of market covariates), $\bar{\mathbf{x}} = \frac{1}{N}\sum_j \mathbf{x}_j$ for the population mean, $\mathbf{w}$ and $\mathbf{v}$ for the treated and control weights, and $z_j \in \{0,1\}$ for whether market j is darkened. The base (`standard`) design solves

$$\min_{\mathbf{w},\mathbf{v},\mathbf{z}} \left\|\bar{\mathbf{x}} - \textstyle\sum_j w_j \mathbf{x}_j\right\|_2^2 + \left\|\bar{\mathbf{x}} - \textstyle\sum_j v_j \mathbf{x}_j\right\|_2^2 \quad \text{s.t.} \quad \textstyle\sum_j w_j = \sum_j v_j = 1,\ \ w_j, v_j \geq 0,\ \ w_j \leq z_j,\ \ v_j \leq 1 - z_j, \tag{4}$$

with the number of darkened markets bounded by $m_{\min} \leq \sum_j z_j \leq m_{\max}$. The constraints $w_j \leq z_j$ and $v_j \leq 1 - z_j$ make a market treated or control but never both, while the objective drives each synthetic unit's pre-launch sales onto the population's. This is a mixed-integer quadratic program (the binary $\mathbf{z}$), which `mlsynth` solves with the open-source SCIP solver (the paper used commercial Gurobi). Once the weights are fixed, the per-week effect is the gap between the two synthetic markets, $\widehat{\tau}_t = \sum_j w_j y_{jt} - \sum_j v_j y_{jt}$. The `MAREX` class also exposes the paper's other objectives through one `design` argument (`weakly_targeted`, `penalized`, `unit_penalized`); we use the base design here for simplicity.

We illustrate on a simulated go-dark test where the true effect is known, so we can check that

MAREX recovers it. Apple wants the incremental revenue its paid media generates across twenty DMAs (the metro regions above) observed over fifty weeks; for the final ten it goes dark in a few markets, where paid media drives a fixed share of sales, and measures the shortfall. The pre-launch sales follow Abadie and Zhao (2026)'s own factor model (seven observed covariates and eleven latent factors, their Section 5), so the markets carry the heterogeneous, correlated histories that make the design problem real. We load the panel and hand it to MAREX:

```
from mlsynth import MAREX
from mlsynth.utils.marex_helpers.plotter import plot_marex

godark = pd.read_csv(find_data("apple_godark_dmas.csv"))   # 20 DMAs x 50 weeks
J, T, T0, T_post = godark["dma"].nunique(), godark["week"].nunique(), 40, 10
covs = ["median_income", "population", "iphone_share", "retail_density",
        "median_age", "broadband_pct", "ad_spend_index"]   # per-DMA market traits

cfg = {"outcome": "sales", "unitid": "dma", "time": "week",
       "T0": T0, "T_post": T_post,        # 40 pre-launch weeks, then a 10-week test
       "m_min": 4, "m_max": 6,            # take four to six markets dark
       "covariates": covs, "standardize": True,   # match on sales and market traits
       "design": "standard", "inference": True, "display_graph": False}

experiment = MAREX({**cfg, "df": godark[["dma", "week", "sales"] + covs]}).fit()
dark = sorted(experiment.treated_units)
w = experiment.globres.treated_weights_agg     # treated (dark) weights, one per DMA
v = experiment.globres.control_weights_agg     # control (lit) weights
Y = godark.pivot(index="dma", columns="week", values="sales").to_numpy()
```

The panel holds 20 DMAs over 50 weeks, the first 40 of them before the test. MAREX reads only that pre-launch window to design, never the post-launch sales, and takes between four and six markets dark, the count left to the optimizer because the paper's own simulations find that allowing more treated units improves the design. On this panel it darkens 6 of the 20 markets and builds a synthetic control for them from the markets that keep spending. A single `fit` does both jobs: the chosen markets' pre-launch sales are untouched by the later blackout, so reading the one panel through to the post-launch weeks turns the design straight into the analysis. The design it returns is the weight vectors $\mathbf{w}$ and $\mathbf{v}$ of Equation 4, one weight per DMA:

```
pd.DataFrame({"Treated weight": w, "Control weight": v},
             index=pd.Index(sorted(godark["dma"].unique()), name="DMA")).round(3)
```

DMA	Treated weight	Control weight
dma00	0.000	0.068
dma01	0.159	0.000
dma02	0.000	0.072
dma03	0.000	0.056
dma04	0.000	0.103

DMA	Treated weight	Control weight
dma05	0.000	0.011
dma06	0.000	0.063
dma07	0.107	0.000
dma08	0.124	0.000
dma09	0.000	0.061
dma10	0.000	0.072
dma11	0.000	0.111
dma12	0.000	0.098
dma13	0.302	0.000
dma14	0.000	0.071
dma15	0.000	0.049
dma16	0.109	0.000
dma17	0.000	0.023
dma18	0.198	0.000
dma19	0.000	0.143

Table 2: The chosen design: treated (dark) and control (lit) weights, one row per DMA. Each market carries weight in exactly one group; the weights within a group sum to one.

A design is only as good as its balance, so before reading any effect we check it. Each DMA carries seven market traits (median income, population, iPhone share, and so on), and MAREX matches on them alongside the pre-launch sales. Figure 5 is a love plot of those traits: it contrasts the raw gap between the darkened and lit markets, the imbalance a naive split would carry, with each synthetic unit against the population, the quantity Equation 4 drives to zero.

```
Z = godark.drop_duplicates("dma").sort_values("dma")[covs].to_numpy()
dark_i = np.where(w > 1e-8)[0]; lit_i = np.where(v > 1e-8)[0]
sd = Z.std(axis=0); popz = Z.mean(axis=0)
raw   = (Z[dark_i].mean(0) - Z[lit_i].mean(0)) / sd      # unweighted dark - lit
smd_t = (w @ Z - popz) / sd                              # synthetic treated vs population
smd_c = (v @ Z - popz) / sd                              # synthetic control vs population
rows = np.arange(len(covs))
fig, ax = plt.subplots(figsize=(6.4, 3.2))
ax.scatter(raw,   rows, color="0.55", marker="s", s=26, label="Dark minus lit, unweighted"
ax.scatter(smd_t, rows, color="#d62728", s=30, label="Synthetic treated vs population")
ax.scatter(smd_c, rows, color="#1f77b4", s=30, label="Synthetic control vs population")
ax.axvline(0, color="black", lw=1)
for x in (-0.1, 0.1):
    ax.axvline(x, color="0.6", ls=":", lw=0.8)
ax.set_yticks(rows); ax.set_yticklabels(covs); ax.set_xlabel("Standardized mean difference
ax.legend(loc="lower right", fontsize=7); plt.tight_layout(); plt.show()
```

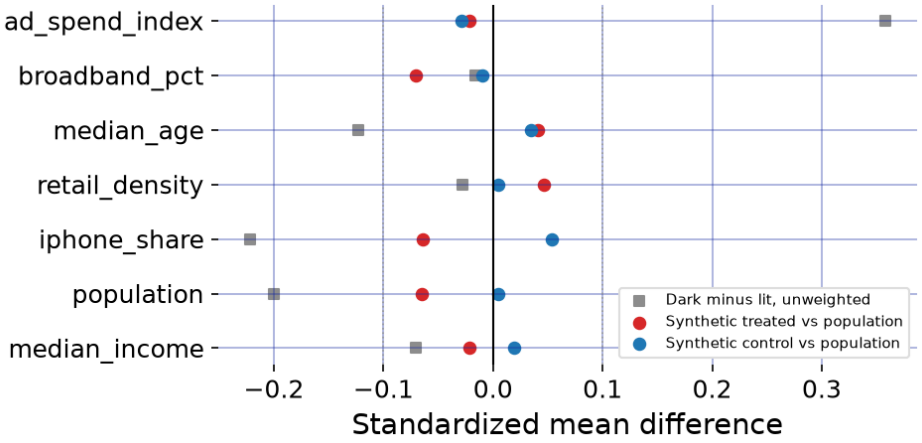

Figure 5: Covariate balance for the chosen design (a love plot), one row per market trait. Grey squares are the raw standardized difference between the darkened and lit markets before weighting; the red and blue dots are the synthetic treated and synthetic control, each against the population. The raw split is off by up to a third of a standard deviation, but MAREX pulls both synthetics onto the population, well inside the dotted 0.1 guides: the design is balanced before the test begins.

With balance established, Figure 6 is MAREX's own diagnostic of the experiment, drawn by the library. The population mean is the thick solid line, the synthetic treated and synthetic control markets (the dark and the lit DMAs) are the red and blue dashed lines, and the faint grey lines are the individual DMAs. Through the fitting window and the held-out blank weeks the two synthetic series sit on top of the population mean, behaving identically before the test, and only after week 40 does the treated series fall away, when paid media in the chosen markets goes dark and their sales drop below the spending counterfactual.

```
plot_marex(experiment, plot_type="prediction", donor_cloud=True, figsize=(7, 3.6))
```

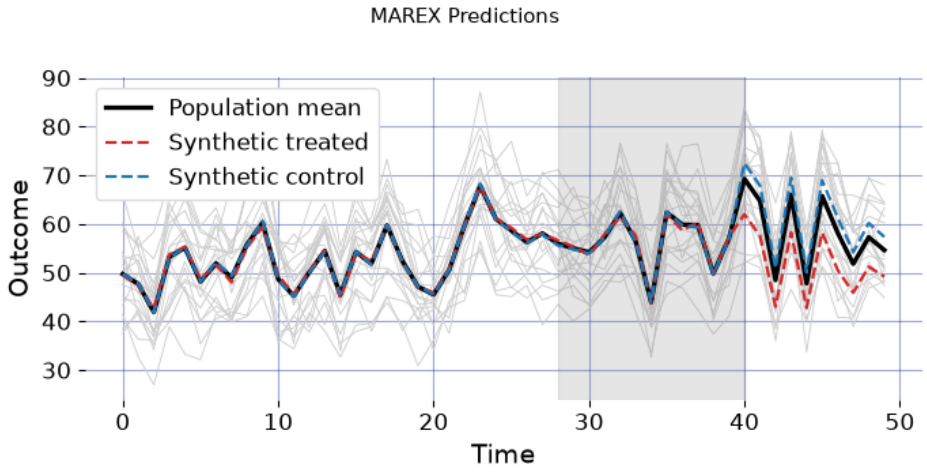

Figure 6: MAREX's own design diagnostic, drawn by the library, for Apple's go-dark test, a reproduction of Abadie and Zhao's Figure 4. The thick solid line is the population mean; the red and blue dashed lines are the synthetic treated and synthetic control markets (the dark and the lit DMAs) the design built from pre-launch sales alone; the faint lines are the individual DMAs. The synthetics track the population through the fitting window and the shaded blank weeks, then the treated series separates once paid media goes dark at week 40. That gap is the sales the dark markets forgo, which is what the advertising had been generating.

Because this is a simulation the true effect is known, and the design recovers it: the mean absolute error of the per-week effect is 0.46, about 5% of the effect's own scale of 9.2, and the two synthetic series track to a pre-launch fit RMSE of 0.58. The result reads like every other estimator's; because MAREX designs an experiment it returns a *design* result, whose `report` is the realized effect and whose `post_fit` carries the diagnostics.

```
pd.Series({
    "report.att":           round(experiment.report.att, 2),   # sales forgone going dark
    "post_fit.ate_percent": round(pf.ate_percent, 1),          # ... as a percentage
    "post_fit.rmse_fit":    round(pf.rmse_fit, 2),             # pre-launch tracking
    "post_fit.p_value":     round(pf.p_value, 3),              # permutation test
    "post_fit.ci_lower":    round(pf.ci_lower, 2),             # conformal band
    "post_fit.ci_upper":    round(pf.ci_upper, 2),
})
```

```
report.att              -9.42
post_fit.ate_percent   -15.40
post_fit.rmse_fit        0.58
post_fit.p_value         0.00
post_fit.ci_lower      -10.66
post_fit.ci_upper       -8.17
dtype: float64
```

Read back into Apple's question, the markets that went dark gave up about 15% of their sales

over the test window. That forgone revenue is exactly the incremental sales the paid media had been generating.

Inference is the other half of the paper's contribution, and `mlsynth` carries it faithfully. Both the permutation p-value for the global null of no effect ($p < 0.001$ on this draw) and the confidence band around each week's effect are built on the *blank* periods, pre-launch weeks deliberately withheld from the fit, where the dark-minus-lit gap is pure noise and so calibrates the size of a null fluctuation. Writing $\mathcal{B}$ for those blank weeks, the band is the post-launch effect plus or minus the $(1-\alpha)$ empirical quantile of the held-out gaps,

$$\widehat{\tau}_t \;\pm\; q_{1-\alpha}, \qquad q_{1-\alpha} = \widehat{Q}_{1-\alpha}(\{\,|\widehat{\tau}_s| : s \in \mathcal{B}\,\}), \tag{5}$$

a split-conformal interval (Lei, G'Sell, Rinaldo, Tibshirani, and Wasserman 2018; Vovk, Gammerman, and Shafer 2005) calibrated on data the weights never saw. The construction is deliberately not the conformal method of Chernozhukov, Wüthrich, and Zhu (2021): where that procedure rearranges over all periods, including the pre-intervention window used to fit the weights, Abadie and Zhao permute only over the blank and post-intervention periods. The payoff is inference that stays valid under serial dependence and non-stationarity, rather than assuming the gaps are i.i.d. over time, which matters because market sales are persistent. Within that same framework `mlsynth` also surfaces a power analysis: an analytical minimum detectable effect, read off the blank-period residuals and inflated for their serial correlation by an AR(1) variance-inflation factor, exposed on `res.post_fit.power` as a headline number and a curve over candidate horizons, so a designer can ask in advance how large an effect the experiment could detect and how long it must run (the derivation is on the estimator's documentation page, https://mlsynth.readthedocs.io/en/latest/marex.html). Design, estimate, permutation test, conformal band, and power curve all fall out of one `fit`, the first time, as far as we know, that this design-and-analyze workflow has been packaged for applied use.

The broader design family

MAREX is one member of a family of experimental-design estimators that `mlsynth` packages together, all sharing the configure-and-fit shape and the standardized result used above. Where MAREX matches the population mean by solving a single mixed-integer program, the design method of Doudchenko *et al.* (2021) takes a target effect size and *chooses* the treated set so that a credible synthetic control will exist afterward, reporting the power of the result; and **LEXSCM**, which has no implementation anywhere else, adapts the synthetic experimental design of Abadie and Zhao (2026), choosing the treated combination lexicographically, validity (pre-treatment fit) ahead of power (a small minimum detectable effect), with the fast small-treated-set algorithm of i Bastida (2022b), ranking combinations by balance and pruning budget-infeasible ones so the search scales where exhaustive enumeration cannot. `mlsynth` also reproduces Meta's industry `GeoLift` geo-experiment walkthrough (Meta Open Source 2024) to within numerical tolerance, shipped as a durable benchmark, so one interface spans the academic and industrial corners of the design literature. Selecting an experiment and, later, analyzing it are two calls against one library rather than two separate tools, and to our knowledge no other package places this design family alongside the full estimation catalog behind a common interface.

5. Summary

Synthetic control has grown from a single method into a sprawling family, and the software has fragmented along with it. We have argued the fragmentation is largely incidental: most of these estimators run the same computation, a treated-unit vector and a donor matrix through an optimization to weights, a counterfactual, and a gap, differing only in how they constrain or denoise that fit. `mlsynth` takes that observation seriously, placing more than forty estimators behind one data contract, one validated configuration-and-fit interface, and one standardized result. The three illustrations show what the consolidation buys. Estimation becomes a genuine comparison: on Abadie and Gardeazabal's Basque study, a nested solver and a provably better bilevel one, which exist together in no other package, reach different synthetic controls from the same data, each by a one-field change against an unchanged data frame. Relaxing the no-interference assumption becomes a measurable sensitivity check: four spillover-aware estimators, otherwise four separate codebases in another language, run across two case studies behind one keyword, their disagreement a quantity to read rather than four outputs to reconcile. And design becomes available at all: MAREX, the first packaged synthetic-control experimental design, chooses an experiment on Abadie and Zhao's simulation and recovers the known effect through the same configure-and-fit interface and standardized result as every estimator before it, opening onto a broader design family the surrounding ecosystem has never packaged for applied users. The value of having all of this in one place is not convenience but capability: a single validated interface makes forty methods comparable and lets one library answer both what an intervention did and how to design the experiment that will measure it.

6. Computational details

The results above were produced with `mlsynth` 1.0.0 on Python 3.13.14, with NumPy 2.5.0 (Harris *et al.* 2020) and pandas 3.0.3, on Linux-6.17.0-1018-azure-x86_64-with-glibc2.39. The convex weight problems are solved with `CVXPY` (Diamond and Boyd 2016). The package and its dependencies are open source; `mlsynth` is available from the Python Package Index via `pip install mlsynth`, with source and issue tracker at https://github.com/jgreathouse9/mlsynth and documentation at https://mlsynth.readthedocs.io.

The library also ships a benchmark suite, a set of small re-runnable scripts that hold every estimator to an outside answer rather than to its own past output. Each benchmark fits one estimator on a fixed dataset and asks whether its numbers match a trusted target to within a stated tolerance, and `mlsynth` obtains that target in one of two ways depending on whether a reference implementation of the method can actually be run. When an authoritative implementation exists and can be built, usually an R package such as `augsynth` or `Synth`, sometimes a research codebase in Python, the benchmark installs that implementation frozen at a specific commit, runs it on the very same input, and compares `mlsynth`'s output to the numbers the reference returns on the spot, quantity by quantity. On the GeoLift geo-experiment walkthrough, for instance, `mlsynth` reproduces a freshly run `augsynth` (the ridge augmented-SCM engine inside Meta's `GeoLift`) to floating point: the cross-validated ridge penalty agrees to about eleven digits, every donor weight to seven, and the post-period effect to four significant figures. When no such implementation can be run, because the method was

only ever published as a paper or its reference code cannot be installed, the benchmark instead targets a number the source paper itself reports, either its headline result on the authors' own data or a cell of its simulation table, recorded once as the value the run must reproduce. The reference cross-checks need the other language's toolchain to be present, so each one skips itself cleanly when that toolchain is missing (for example in routine continuous integration, which carries no R) and runs only where the reference is installed; the paper-number checks need nothing beyond `mlsynth` and run everywhere.

Pinning each estimator to an outside answer in this way, a live reference or a printed paper value, is what keeps it from drifting unnoticed. Expected numbers that a developer once typed into a test slowly rot as a reference is updated or an estimator is refactored, yet the test keeps passing against a stale target; here a reference is recomputed from frozen source on every run and a paper value is a published fact the method must keep matching, so a regression in either `mlsynth` or an updated reference surfaces as a failed benchmark rather than a quiet disagreement the paper never notices. Freezing each reference at an exact commit, rather than tracking its moving development tip, makes the comparison a stable, dated checkpoint that runs the same code every time and is refreshed only by deliberately bumping the pinned commit and re-recording the expected numbers. The install scripts, the re-run harness, and the full replication code are provided as the replication materials accompanying this article and in the `benchmarks` suite of the source repository.

References

Abadie A (2021). "Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects." *Journal of Economic Literature*, **59**(2), 391–425. doi:10.1257/jel.20191450.

Abadie A, Diamond A, Hainmueller J (2010). "Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program." *Journal of the American Statistical Association*, **105**(490), 493–505. doi:10.1198/jasa.2009.ap08746.

Abadie A, Diamond A, Hainmueller J (2011). "**Synth**: An R Package for Synthetic Control Methods in Comparative Case Studies." *Journal of Statistical Software*, **42**(13), 1–17. doi:10.18637/jss.v042.i13.

Abadie A, Diamond A, Hainmueller J (2015). "Comparative Politics and the Synthetic Control Method." *American Journal of Political Science*, **59**(2), 495–510. doi:10.1111/ajps.12116.

Abadie A, Gardeazabal J (2003). "The Economic Costs of Conflict: A Case Study of the Basque Country." *American Economic Review*, **93**(1), 113–132. doi:10.1257/000282803321455188.

Abadie A, Zhao J (2026). "Synthetic Controls for Experimental Design." 2108.02196, URL https://arxiv.org/abs/2108.02196.

Agarwal A, Shah D, Shen D (2026). "Synthetic Interventions: Extending Synthetic Controls to Multiple Treatments." *Operations Research*, **74**(2), 840–859. doi:10.1287/opre.2025.1590.

Amjad M, Shah D, Shen D (2018). "Robust Synthetic Control." *Journal of Machine Learning Research*, **19**(22), 1–51.

Arkhangelsky D, Athey S, Hirshberg DA, Imbens GW, Wager S (2021). "Synthetic Difference-in-Differences." *American Economic Review*, **111**(12), 4088–4118. doi:10.1257/aer.20190159.

Athey S, Bayati M, Doudchenko N, Imbens G, Khosravi K (2021). "Matrix Completion Methods for Causal Panel Data Models." *Journal of the American Statistical Association*, **116**(536), 1716–1730. doi:10.1080/01621459.2021.1891924.

Bai J (2009). "Panel Data Models with Interactive Fixed Effects." *Econometrica*, **77**(4), 1229–1279. doi:10.3982/ECTA6135.

Becker M, Klößner S (2018). "Fast and Reliable Computation of Generalized Synthetic Controls." *Econometrics and Statistics*, **5**, 1–19. doi:10.1016/j.ecosta.2017.08.002.

Ben-Michael E, Feller A, Rothstein J (2021). "The Augmented Synthetic Control Method." *Journal of the American Statistical Association*, **116**(536), 1789–1803. doi:10.1080/01621459.2021.1929245.

Brodersen KH, Gallusser F, Koehler J, Remy N, Scott SL (2015). "Inferring Causal Impact Using Bayesian Structural Time-Series Models." *The Annals of Applied Statistics*, **9**(1), 247–274. doi:10.1214/14-AOAS788.

Cao J, Dowd C (2023). "Estimation and Inference for Synthetic Control Methods with Spillover Effects." *arXiv preprint arXiv:1902.07343*.

Cattaneo MD, Feng Y, Palomba F, Titiunik R (2025). "**scpi**: Uncertainty Quantification for Synthetic Control Methods." *Journal of Statistical Software*, **113**(2), 1–38. doi:10.18637/jss.v113.i02.

Cattaneo MD, Feng Y, Titiunik R (2021). "Prediction Intervals for Synthetic Control Methods." *Journal of the American Statistical Association*, **116**(536), 1865–1880. doi:10.1080/01621459.2021.1979561.

Chernozhukov V, Wüthrich K, Zhu Y (2021). "An Exact and Robust Conformal Inference Method for Counterfactual and Synthetic Controls." *Journal of the American Statistical Association*, **116**(536), 1849–1864. doi:10.1080/01621459.2021.1920957.

Chernozhukov V, Wuthrich K, Zhu Y (2025). "Debiasing and t-tests for synthetic control inference on average causal effects." 1812.10820, URL https://arxiv.org/abs/1812.10820.

Clarke D, Pailañir D, Athey S, Imbens G (2023). "Synthetic Difference in Differences Estimation." *arXiv preprint arXiv:2301.11859*. doi:10.48550/arXiv.2301.11859.

Colvin S, Pydantic Contributors (2024). ***pydantic**: Data Validation Using Python Type Hints*.

Di Stefano R, Mellace G (2024). "The inclusive Synthetic Control Method." 2403.17624, URL https://arxiv.org/abs/2403.17624.

Diamond S, Boyd S (2016). "**CVXPY**: A Python-Embedded Modeling Language for Convex Optimization." *Journal of Machine Learning Research*, **17**(83), 1–5.

Doudchenko N, Khosravi K, Pouget-Abadie J, Lahaie S, Lubin M, Mirrokni V, Spiess J, Imbens G (2021). "Synthetic Design: An Optimization Approach to Experimental Design with Synthetic Controls." *Advances in Neural Information Processing Systems*, **34**.

Dunford E (2021). "**tidysynth**: A Tidy Implementation of the Synthetic Control Method." https://github.com/edunford/tidysynth.

Engelbrektson O (2020). "**SyntheticControlMethods**: A Python Package for Causal Inference Using Synthetic Controls." https://github.com/OscarEngelbrektson/SyntheticControlMethods.

Grossi G, Mariani M, Mattei A, Lattarulo P, Öner Ö (2025). "Direct and spillover effects of a new tramway line on the commercial vitality of peripheral streets: a synthetic-control approach." *Journal of the Royal Statistical Society Series A: Statistics in Society*, **188**(1), 223–240. doi:10.1093/jrsssa/qnae032.

Harris CR, Millman KJ, van der Walt SJ, *et al.* (2020). "Array Programming with NumPy." *Nature*, **585**, 357–362. doi:10.1038/s41586-020-2649-2.

Holland PW (1986). "Statistics and Causal Inference." *Journal of the American Statistical Association*, **81**(396), 945–960. doi:10.1080/01621459.1986.10478354.

Hsiao C, Ching HS, Wan SK (2012). "A Panel Data Approach for Program Evaluation: Measuring the Benefits of Political and Economic Integration of Hong Kong with Mainland China." *Journal of Applied Econometrics*, **27**(5), 705–740. doi:10.1002/jae.1230.

i Bastida JV (2022a). "Predictor Selection for Synthetic Controls." 2203.11576, URL https://arxiv.org/abs/2203.11576.

i Bastida JV (2022b). "Synthetic Experimental Design for a UBI Pilot Study." Working paper, URL https://ivalua.cat/sites/default/files/2023-03/Vives-i-Bastida_2022_anon.pdf.

Imbens GW, Rubin DB (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, New York. doi:10.1017/CBO9781139025751.

Kellogg M, Mogstad M, Pouliot GA, Torgovitsky A (2021). "Combining Matching and Synthetic Control to Tradeoff Biases from Extrapolation and Interpolation." *Journal of the American Statistical Association*, **116**(536), 1804–1816. doi:10.1080/01621459.2021.1979562.

Lei J, G'Sell M, Rinaldo A, Tibshirani RJ, Wasserman L (2018). "Distribution-Free Predictive Inference for Regression." *Journal of the American Statistical Association*, **113**(523), 1094–1111. doi:10.1080/01621459.2017.1307116.

Lei L, Sudijono T (2025). "Inference for Synthetic Controls via Refined Placebo Tests." 2401.07152, URL https://arxiv.org/abs/2401.07152.

Liu J, Tchetgen Tchetgen E, Varjão C (2024). "Proximal Causal Inference for Synthetic Control with Surrogates." In S Dasgupta, S Mandt, Y Li (eds.), *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pp. 730–738. PMLR. URL https://proceedings.mlr.press/v238/liu24a.html.

Malo P, Eskelinen J, Zhou X, Kuosmanen T (2024). "Computing Synthetic Controls Using Bilevel Optimization." *Computational Economics*, **64**(2), 1113–1136. doi:10.1007/s10614-023-10471-7.

Melnychuk A (2024). "Synthetic Controls with spillover effects: A comparative study." 2405.01645, URL https://arxiv.org/abs/2405.01645.

Meta Open Source (2024). "GeoLift: An End-to-End Geo-Experimentation Methodology." https://github.com/facebookincubator/GeoLift.

Park C, Tchetgen EJT (2025). "Single proxy synthetic control." *Journal of Causal Inference*, **13**(1), 20230079. doi:10.1515/jci-2023-0079.

Peng T, Agarwal A, Shah D (2022). "**causaltensor**: A Python Package for Causal Inference with Tensor/Matrix Completion." https://github.com/TianyiPeng/causaltensor.

PyMC Labs (2024). "**CausalPy**: A Python Package for Causal Inference in Quasi-Experimental Settings." https://github.com/pymc-labs/CausalPy.

Qiu H, Shi X, Miao W, Dobriban E, Tchetgen Tchetgen E (2024). "Doubly robust proximal synthetic controls." *Biometrics*, **80**(2), ujae055. ISSN 1541-0420. doi:10.1093/biomtc/ujae055. https://academic.oup.com/biometrics/article-pdf/80/2/ujae055/58031293/ujae055.pdf, URL https://doi.org/10.1093/biomtc/ujae055.

Qiu J, Jammalamadaka SR, Ning N (2018). "Multivariate Bayesian Structural Time Series Model." *Journal of Machine Learning Research*, **19**(68), 1–33.

Robbins MW, Davenport S (2021). "**microsynth**: Synthetic Control Methods for Disaggregated and Micro-Level Data in R." *Journal of Statistical Software*, **97**(2), 1–31. doi:10.18637/jss.v097.i02.

Rubin DB (1974). "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." *Journal of Educational Psychology*, **66**(5), 688–701. doi:10.1037/h0037350.

Sevigny EL, Greathouse J, Medhin DN (2023). "Health, safety, and socioeconomic impacts of cannabis liberalization laws: An evidence and gap map." *Campbell Systematic Reviews*, **19**(4), e1362. doi:https://doi.org/10.1002/cl2.1362. https://onlinelibrary.wiley.com/doi/pdf/10.1002/cl2.1362, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cl2.1362.

Shi X, Li KQ, Yu M, Miao W, Kuchibhotla AK, Hu M, Tchetgen ET (2026). "Theory for Identification and Inference with Synthetic Controls: A Proximal Causal Inference Framework." *Journal of the American Statistical Association*, **0**(0), 1–13. doi:10.1080/01621459.2026.2639734.

Shi Z, Huang J (2023). "Forward-Selected Panel Data Approach for Program Evaluation." *Journal of Econometrics*, **234**(2), 512–535. doi:10.1016/j.jeconom.2021.04.009.

Vega-Bayo A (2015). "An R Package for the Panel Approach Method for Program Evaluation: **pampe**." *The R Journal*, **7**(2), 105–121. doi:10.32614/RJ-2015-029.

Vovk V, Gammerman A, Shafer G (2005). *Algorithmic Learning in a Random World*. Springer, New York.

Xu Y (2017). "Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models." *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.

Xu Y, Zhou Q (2025). "Bayesian Synthetic Control with a Soft Simplex Constraint." *arXiv preprint arXiv:2505.00006*.

Yan G, Chen Q (2022). "**rcm**: A Command for the Regression Control Method." *The Stata Journal*, **22**(4), 842–883. doi:10.1177/1536867X221140960.

Yan G, Chen Q (2023). "**synth2**: Synthetic Control Method with Placebo Tests, Robustness Test, and Visualization." *The Stata Journal*, **23**(3), 597–624. doi:10.1177/1536867X231195278.

Affiliation:

Jared Greathouse
Department of Economics, Andrew Young School of Policy Studies
Georgia State University
Atlanta, GA, United States
E-mail: jgreathouse3@student.gsu.edu
URL: https://jgreathouse9.github.io/

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd